

DATA COMMUNITY TRÓJMIASTO • 31. SPOTKANIE

Trójkąty, kaczki i góry lodowe

Przegląd technologii Data Lakehouse na 2026

Gdańsk, 14 stycznia 2026

25+

lat na rynku

**Klienci w 15
krajach**

800+

ludzi na pokładzie



74

Net Promoter
Score

Data Solutions

by:  Future Processing

3

wiodące chmury

Akceleratorzy - Data Platform

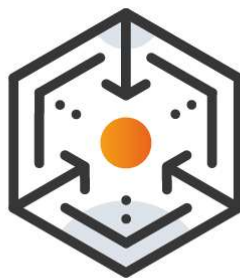
Usługi - FinDataOps

Projekty - Green Field i Migracje

Rozwój - szkółki i upskilling

60+

inżynierów danych



8+

lat działalności DS

Dane vs Opinie

If we have data, let's look at data.

If all we have are opinions, let's go with mine.

- James Barksdale, Netscape CEO 1995-1999

DLACZEGO TO WAŻNE

Dostęp do danych w odpowiednim momencie determinuje jakość decyzji biznesowych.

Bez danych - decyduje szef. Z danymi - decydują fakty.

Plan na dzisiaj

CZĘŚĆ 1

5 filarów platform danych

Czego chcemy od systemów przetwarzania danych?

CZĘŚĆ 2

Produkty Data...

Database → Warehouse →
Lake → Lakehouse

CZĘŚĆ 3

Technologie Lakehouse

Delta • Iceberg • Hudi •
DuckLake

CZĘŚĆ 4

Benchmarki & Porównania

Twarde dane, nie opinie

CZĘŚĆ 5

Dokąd zmierzamy w 2026?

Trendy i predykcje

5 Filarów platform danych

1

Przechowywanie

Czy jest trwałe? Skalowalne?
Opłacalne?

2

Przetwarzanie

Batch? Streaming? Jakie
opóźnienie?

3

Dostęp

SQL? API? Raporty? Kto
konsumuje?

4

Bezpieczeństwo

RBAC, row-level, maskowanie,
szyfrowanie

5

Governance

Czy możemy ufać danym?
Stewardship, lineage, quality

PRODUKT 1

Database

System przechowywania i pobierania ustrukturyzowanych danych z gwarancją spójności

KIEDY

1970

KTO

Edgar F. Codd, IBM San Jose

"A Relational Model of Data for Large Shared Data Banks"

GDZIE

Oracle • MySQL • PostgreSQL • SQL Server • MariaDB • SQLite • IBM Db2

DLACZEGO POWSTAŁ

Zastąpienie hierarchii (drzewa) matematycznym fundamentem relacji

Eliminacja redundancji danych

Umożliwienie ad-hoc queries przez SQL

Gwarancja spójności danych (ACID)

DATABASE

Baza danych – 5 filarów

1. PRZECHOWYWANIE

Row-oriented, tightly-coupled z compute, sztywna schema, gwarancja ACID

2. PRZETWARZANIE

Transakcje (OLTP), single-node

3. DOSTĘP

SQL, jeden silnik

4. BEZPIECZEŃSTWO ✓

Wbudowane RBAC, uprawnienia na poziomie wierszy/kolumn, full opcja, dojrzałe

5. GOVERNANCE ✓

Integralność referencyjna (wymuszanie schematu on write), constrainty

GLÓWNE ZADANIE

OLTP - szybkie, spójne operacje transakcyjne

DATABASE

Mocne strony i ograniczenia

✓ MOCNE STRONY

Transakcje ACID - atomicity, consistency, isolation, durability

Milisekundowe opóźnienie - dla zapisu i odczytu rekordów

Integralność referencyjna - wymuszanie schematu

50+ lat toolingu, ekspertyzy, stabilności

✗ OGRANICZENIA

Row-oriented = wolne agregacje na dużych zbiorach

Tylko skalowanie wertykalne (scale-up, nie scale-out)

Drogie przy skali analitycznej

Nie dla danych nieustrukturyzowanych

→ **LUKA**

Analityka na dużych zbiorach historycznych

→ **Data Warehouse**

PRODUKT 2

Data Warehouse

Hurtownia - scentralizowane repozytorium zintegrowanych danych z wielu źródeł, zoptymalizowane dla zapytań analitycznych

KIEDY

1990

KTO

Bill Inmon - ojciec DW
Ralph Kimball - modelowanie wymiarowe

GDZIE

Teradata • Oracle DW • IBM Db2 Warehouse • MS SQL Server Analysis Services

Star schema, snowflake schema, fact/dimension tables

DLACZEGO POWSTAŁ

Oddzielenie workloadów analitycznych od systemów transakcyjnych

Integracja danych z wielu systemów źródłowych w jedno źródło prawdy

Analiza historyczna (time-variant data)

Wsparcie Business Intelligence i raportowania

DEFINICJA INMONA

Subject-oriented • Integrated • Time-variant • Non-volatile

Jak Data Warehouse realizuje 5 filarów?

1. PRZECIĄGNIĘCIE ✓

Column-oriented = zoptymalizowane dla odczytu + kompresji,
tightly-coupled z compute

2. PRZETWARZANIE

ETL (Extract-Transform-Load), batch-oriented, scheduled jobs (nightly/weekly)

3. DOSTĘP ✓

SQL zoptymalizowany dla agregacji, jeden silnik

4. BEZPIECZEŃSTWO ✓

Enterprise-grade, odziedziczone po świecie database

5. GOVERNANCE ✓

Star/snowflake schemas, metadata catalogs, defined ownership

GLÓWNE ZADANIE

OLAP - zapytania analityczne na dużych zbiorach historycznych

Mocne strony i ograniczenia

✓ MOCNE STRONY

Column-oriented = szybkie agregacje (SUM, AVG, COUNT)

Single source of truth dla business reporting

Schema wymusza jakość i spójność danych

Mature BI tooling (Cognos, MicroStrategy)

✗ OGRANICZENIA

Drogi storage - proprietary, on-prem

Szywna schema - zmiany są wolne i bolesne

ETL w batchach - dane opóźnione godziny/dni

Nie dla danych nieustrukturyzowanych

→ LUKA

Przechowuj wszystko tanio, obsługuj dowolne typy danych, elastyczny schemat

→ Data Lake

PRODUKT 3

Data Lake

Scentralizowane repozytorium przechowujące wszystkie dane w surowym formacie - structured, semi-structured i unstructured

KIEDY

2010

KTO

James Dixon, CTO Pentaho

Blog: "Pentaho, Hadoop, and Data Lakes"

GDZIE

Hadoop HDFS (2006-2015)

Cloud: AWS S3 • Azure ADLS • GCS

Processing: Spark, Hive, Presto | Formats: Parquet, ORC, Avro

DLACZEGO POWSTAŁ

Przechowuj wszystko tanio bez upfront schema design

Obsługa typów danych których DW nie obsługuje (logi, JSON, obrazy)

Schema-on-read zamiast schema-on-write

Wsparcie Data Science i ML workloads

Ominięcie ETL bottleneck - load first, figure out later

METAFORA DIXONA

"DW = bottled water (cleansed, packaged). Lake = natural body of water"

DATA LAKE

Jak Data Lake realizuje 5 filarów?

1. PRZECHOWYWANIE ✓✓

Tanie, skalowalne, dowolny format, oddzielone od compute

2. PRZETWARZANIE

Batch (Spark, Hive), trochę streaming, schema-on-read

3. DOSTĘP X

Wiele silników (Spark, Presto, Hive), brak standardu

4. BEZPIECZEŃSTWO X

Bolt-on, niespójne, file/folder-level ACLs

5. GOVERNANCE X X

Często brakuje - brak schema enforcement, lineage, katalogu

GŁÓWNE ZADANIE

Przechowuj wszystko tanio, przetwarzaj elastycznie

DATA LAKE

Mocne strony i ograniczenia

✓ MOCNE STRONY

Ekstremalnie tani storage - kilka centów za GB/msc

Dowolny typ danych - structured, semi, unstructured

Decoupled storage i compute

ML/AI friendly - surowe dane dla data scientistów

✗ OGRANICZENIA

Brak ACID łatwo popsuć dane

Brak schema enforcement - nie ma kontroli nad tym co wpada

Słaba wydajność - małe pliki, brak indeksów

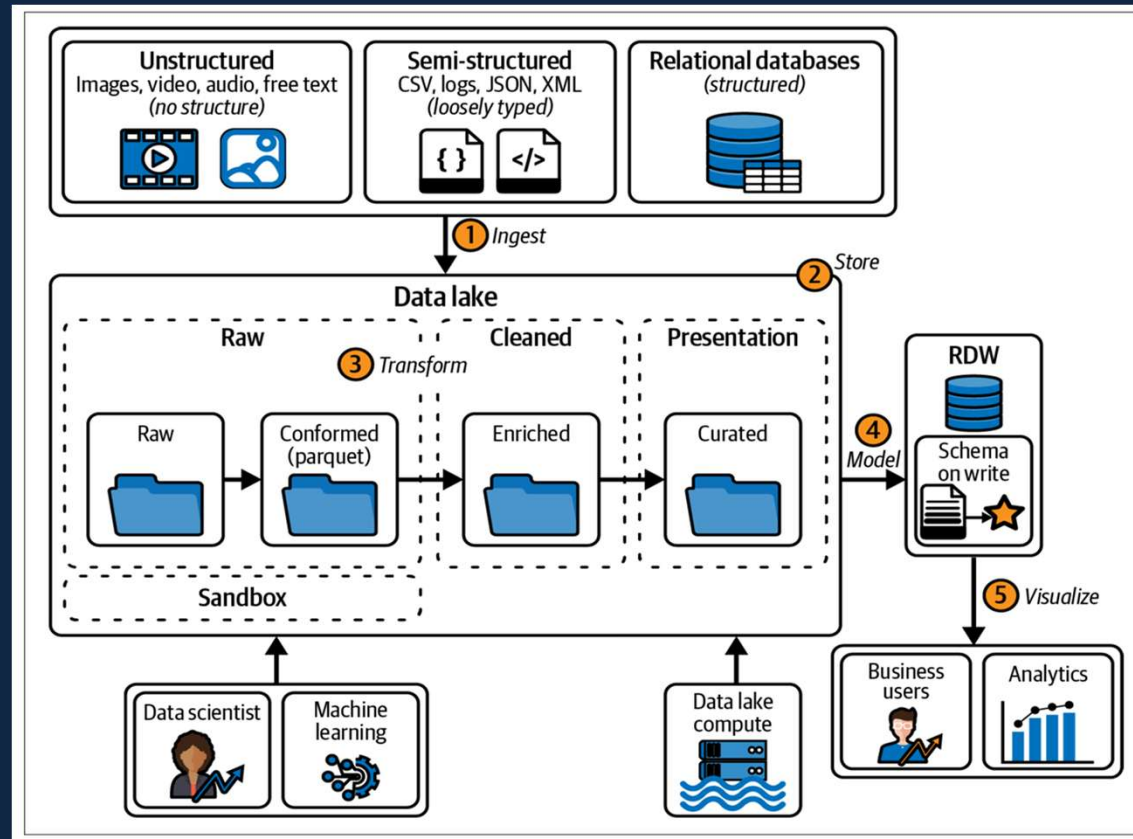
Koszmar Governanceu → DATA SWAMP

→ LUKA

Jak połączyć elastyczność lake'a z niezawodnością warehouse'a

→ **Modern Data Warehouse**

Modern Data Warehouse



Hybryda: Data Lake (staging) + Relational DW (serving) - dwa systemy współpracujące

Architektura hybrydowa: Lake + Warehouse

KIEDY / KTO

~2015-2016 - James Serra (Microsoft)

Wzorzec sformalizowany gdy organizacje zmagaly się z integracją lake + warehouse

GDZIE

Azure: ADLS + Synapse

AWS: S3 + Redshift

GCP: Cloud Storage + BigQuery

5 etapów: Ingest → Store → Transform → Model → Visualize

DLACZEGO POWSTAŁ

Elastyczność + niezawodność - lake + warehouses

Masowe zbieranie surowych danych – staging w lake

RDW dla danych curated z analizą w klasycznym SQL

Wsparcie zarówno Data Science (lake) jak i BI (warehouse)

Zamiast wybierać – weź oba lake + warehouse

⚠ MDW ≠ produkt (jak Snowflake). To wzorzec architektury.

Jak MDW realizuje 5 filarów?

1. PRZECHOWYWANIE ✓

Tani object storage (lake) + relational storage (DW) dla danych curated, decoupled

3. DOSTĘP ✓

Wiele sposobów: Spark/Presto itp. na lake + SQL na warehouse

5. GOVERNANCE

Podzielone: surowe dane w lake (mniejszy governance) + curated w warehouse (większy governance)

2. PRZETWARZANIE

ETL/ELT między lake a warehouse, batch-oriented, możliwy streaming do lake

4. BEZPIECZEŃSTWO

Podzielone: lake (ACL do plików) + warehouse security (RBAC, row-level)

GLÓWNE ZADANIE

Połączenie elastyczności lake'a i niezawodności warehouse'a

Mocne strony i ograniczenia

✓ MOCNE STRONY

Surowe dane zachowane w lake dla DS, a **dane curated** w DW dla BI

Elastyczne skalowanie storage i compute - cloud native

Znajomy interface SQLowy (dla osób doświadczonych z RDW)

Wszechstronność - dane strukturalne i niestrukturalne

X OGRANICZENIA

Dwa systemy - 2x złożoność zarządzania 2 systemami i pipeline'ami między nimi

Duplikacja danych - płacimy za to samo dwa razy + ryzyko bagna

Synchronizacja to wyzwanie - lake i DW mogą się rozjechać

Opóźnienie danych - DW ma zawsze dane starsze niż lake

→ LUKA

Czy możemy mieć benefity obu bez utrzymywania dwóch systemów?

→ **Data Lakehouse**

PRODUKT 5

Data Lakehouse

KIEDY

2020

KTO

Databricks

Blog: "What Is a Lakehouse?"

GDZIE (TECHNOLOGIE)

Formaty tabeli: Delta Lake, Iceberg, Hudi, Duck Lake

Platformy: Databricks, Snowflake, AWS, Azure, GCP

Silniki obliczeniowe: Spark, Trino, Dremio, DuckDB

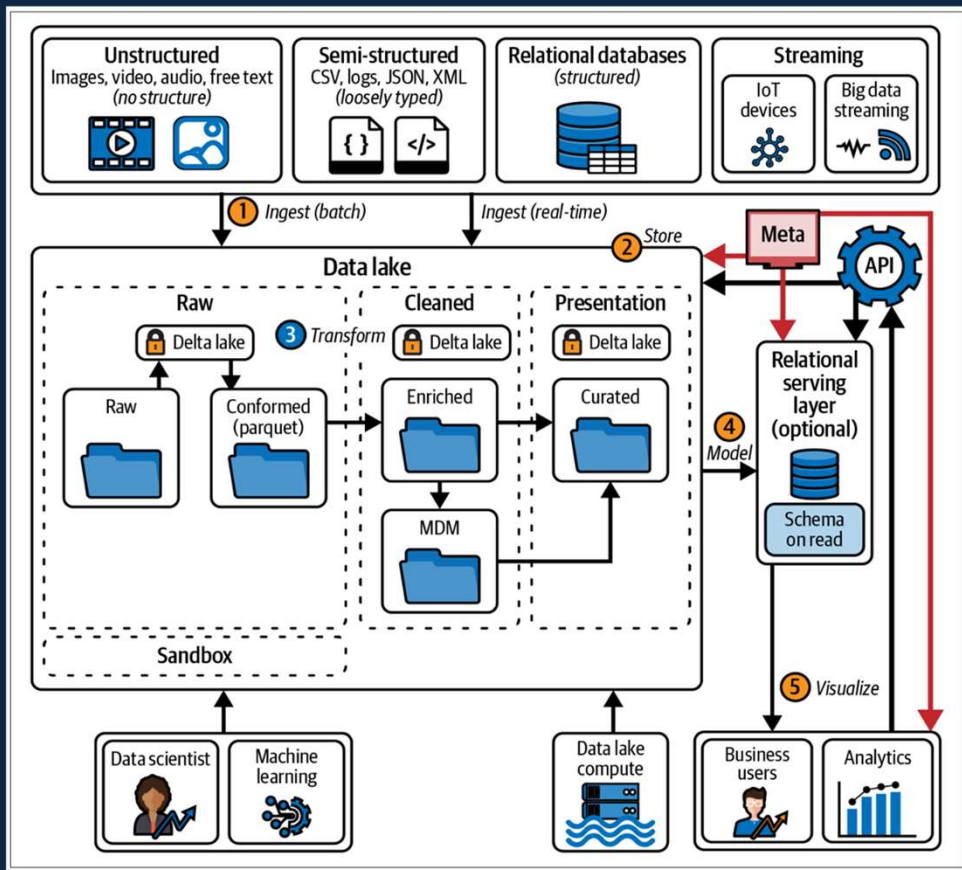
Katalogi: Unity, Polaris, Glue, Hive, Postgres

DLACZEGO POWSTAŁ

Eliminacja problemu dwóch systemów z MDWH

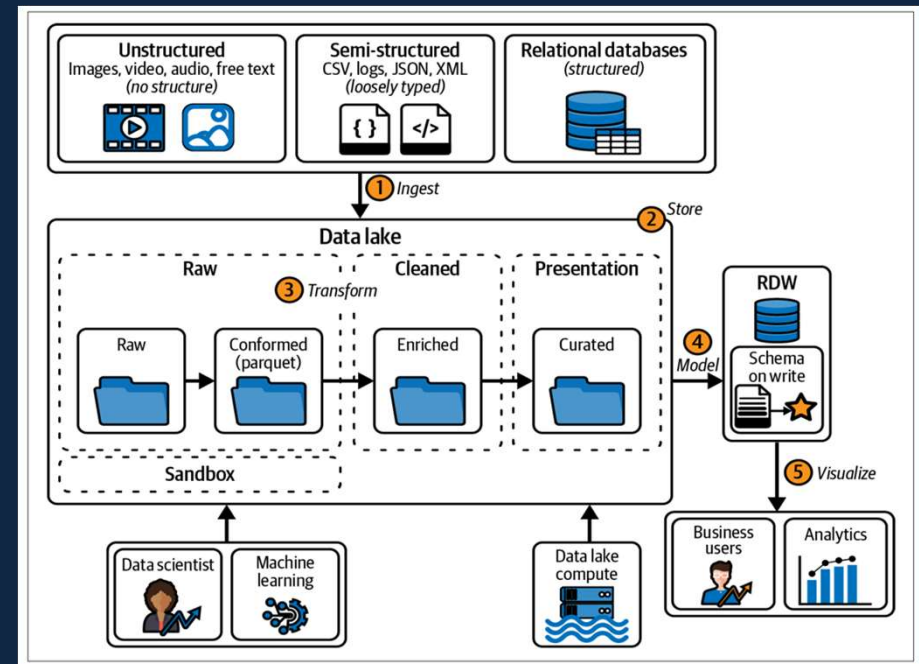
BI i ML na tych samych danych

Data Lakehouse



Kluczowa różnica vs MDW:

Dane zostają w Lake (Delta/Parquet), brak kopiowania do RDW.



Kluczowy element: Pliki jako Tabele

TABLE FORMAT



Strażnik plików

Warstwa metadanych zamienia "pliki" w "tabele"

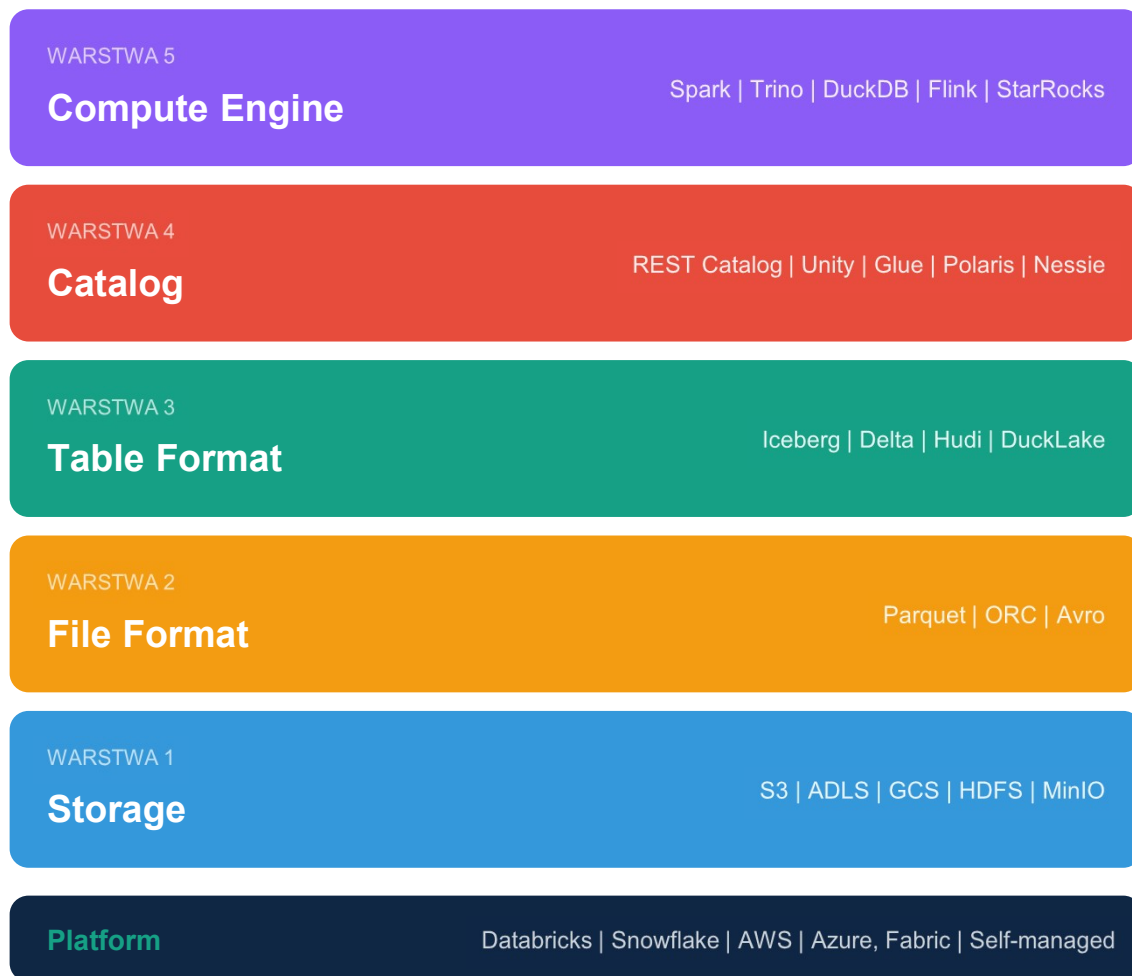
ACID transactions – silna gwarancja danych

Schema enforcement + evolution - dane muszą pasować do schemy, która może się zmieniać w czasie

Time travel - commit = snapshot, można łatwo wrócić do przeszłości

Formaty: Delta Lake • Apache Iceberg • Apache Hudi • DuckLake

Architektura Lakehouse: Warstwy i relacje



ENGINE → wykonuje zapytania

Przetwarza SQL, optymalizuje plany, skaluje compute. Może być wymieniony bez zmiany danych.

CATALOG → rejestruje tabele

Śledzi lokalizacje, schematy, uprawnienia. Umożliwia discovery i governance.

TABLE FORMAT → organizuje dane

ACID, time travel, schema evolution. Metadane + konwencje na plikach.

FILE FORMAT → koduje kolumny

Kompresja, columnar layout, statistics. Parquet = de facto standard.

STORAGE → przechowuje pliki

Object storage, tani, skalowalny. Decoupled od compute.

Jak Lakehouse realizuje 5 filarów?

1. PRZECHOWYWANIE

Tani storage chmurowy (S3/ADLS/GCS), otwarty format plików (Parquet), decoupled

2. PRZETWARZANIE

ACID transactions, schema enforcement, możliwe połączenie batch + streaming

3. DOSTĘP

Multi-engine (Spark, Trino, Snowflake, DuckDB), SQL standard

4. BEZPIECZEŃSTWO

Stale się poprawia - katalogi oferują RBAC, row/column-level access

5. GOVERNANCE

Table formats: versioning, lineage, schema evolution, time travel

GLÓWNE ZADANIE

Elastyczność Lake + niezawodność Warehouse w jednym systemie

Mocne strony i ograniczenia

✓ MOCNE STRONY

ACID transactions na tanim object storage

Open formats - brak vendor lock-in

Schema enforcement + schema evolution

Time travel - query historical, rollback

✗ OGRANICZENIA

Format tabeli to nie platforma – trzeba dobierać komponenty

Wydajność zależy od konfiguracji - compaction (łączenie plików), sizing (partition, file)

Complexity - więcej moving parts

Streaming nadal trudniejszy niż batch

→ Jak jest to zaimplementowane?

Technologie Lakehouse: **Iceberg** • **Delta** • **Hudi** • **DuckLake**

TABLE FORMAT 1

Apache Hudi

Hadoop Upserts Deletes and Incrementals

KIEDY

2016

Apache TLP 2020

KTO

Vinoth Chandar, Uber

GDZIE

Uber, TikTok, Walmart, Amazon, ByteDance

Processing: Spark, Flink | Managed: AWS EMR, Onehouse

DLACZEGO POWSTAŁ - PROBLEM UBERA

Miliony tripów, każdy trip się aktualizuje (status, lokacja, cena)

Przewidywanie pozostałego czasu, dynamiczne ceny, znajdowanie kierowców - wszystko near real-time

Problem: Pliki Parquet są immutable (niezmienialne), nie można update'ować wierszy

Tradycyjnie: przepisz całą partycję. Przy dużej skali (PB) = niemożliwe

→ **Rozwiązanie: Record-level upserts z indeksami dla szybkiego dostępu**

APACHE HUDI

Kluczowe cechy

TYPY TABEL

CoW: Copy-on-Write – pliki przepisywane podczas zapisu

MoR: Merge-on-Read – append delty, a merge dopiero podczas odczytu

★ CHARAKTERYSTYCZNA CECHA: INCREMENTAL QUERIES

Możliwe zapytania tylko o to co się zmieniło od timestamp X - o wiele szybciej niż full scan

STREAMING

Stworzony z myślą o streamingu

INDEXING

Wbudowane indeksy dla szybkiego odnajdywania rekordów

TRANSACTIONS

ACID dzięki kontroli dostępu bazującej na timeline

COMPACTION

Automatic background compaction for MoR tables

UNIQUE CONCEPT: TIMELINE

Każda akcja (commit, compaction, clean) rejestrowana na timeline → time travel, incremental consumption, rollback

APACHE HUDI

Mocne strony i ograniczenia

✓ MOCNE STRONY

Najszybsze update'y na poziomie rekordów

Natywne wsparcie dla incremental queries

Elastyczność zapisu lub odczytu (CoW/MoR)

Mature streaming - minute-level freshness

✗ OGRANICZENIA

Wysoka złożoność – dodatkowy wybór CoW vs MoR

Mało silników - Głównie dla Sparka – Flink z problemami

Małe community – istotne dla Open Source

NAJLEPSZY DO

CDC • Streaming • Upsert-heavy workloads

Kafka + CDC + wysoka świeżość → Hudi

TABLE FORMAT 2

Delta Lake

ACID transactions dla data lakes

KIEDY

2017/2019

Created/Open-sourced

KTO

Databricks

Spark creators

GDZIE

Databricks (native), Microsoft Fabric (default)

60%+ Fortune 500 (via Databricks) [2024]

UniForm → generates Iceberg/Hudi metadata

DLACZEGO POWSTAŁ - PROBLEM DATABRICKS

Klienci budowali data lake na Spark, ale ciągle powstawało bagno

Głównie przez brak ACID i rygorystycznej schemy

oraz brak transakcyjności, rollbacków i time travela

→ **Rozwiązanie: Metadata layer na Parquet → ACID + schema + time travel**

DELTA LAKE

Kluczowe cechy

STRUKTURA METADANYCH

Transaction log (`_delta_log/`) - JSON commits + Parquet checkpoints co 10 commits

UNIQUE: UNIFORM

Auto-generates Iceberg/Hudi metadata → write Delta, read as Iceberg

DELETION VECTORS

Mark rows deleted bez rewrite file - deletes 10x szybciej

TRANSAKCJE

Optimistic concurrency control; serializable isolation

LIQUID CLUSTERING

Dynamic clustering (zastępuje partycjonowanie) - brak wyboru partition columns upfront

PERFORMANCE

Z-ordering, data skipping, auto-optimize, Photon engine

BEST SPARK INTEGRATION

Created by Spark creators, optimized together. Features jak Photon engine acceleration tylko z Delta.

DELTA LAKE

Mocne strony i ograniczenia

✓ MOCNE STRONY

Najlepsza integracja ze Sparkiem – stworzony przez tych samych ludzi

Przetestowany w boju – przez największych klientów

Dojrzałe funkcjonalności wspomagające wydajność

UniForm – zapisz jako Delta, odczytaj jako Iceberg

✗ OGRANICZENIA

Optymalnie działa w Databricks, a OSS ma mniej funkcjonalności

Spark-first – ograniczone wsparcie innych silników (ale nadal duże)

Liniowy TX log – być może bottleneckem wydajności

NAJLEPSZY DLA

Databricks • Spark • Microsoft Fabric

Jesteś na Databricks/Fabric lub masz dużo Sparka? → Delta

TABLE FORMAT 3

Apache Iceberg

Open table format dla huge analytic tables

KIEDY

2017

Apache TLP 2020

KTO

Ryan Blue, Dan Weeks

Netflix

GDZIE (EVERYONE!)

Netflix, Apple, LinkedIn, Airbnb, Adobe, Stripe

Clouds: Snowflake, AWS S3 Tables, BigQuery, Databricks

Engines: Spark, Flink, Trino, Dremio, StarRocks, DuckDB

DLACZEGO POWSTAŁ - PROBLEM NETFLIX

Petabajty na Hive - race conditions w partition discovery (jednoczesny odczyt i zapis partycji)

Potrzeba atomic transactions na olbrzymich tabelach

Vendor lock in – Tabele Hive użyteczne tylko w ekosystemie Hadoop/Spark

Schema evolution psuła zapytania

APACHE ICEBERG

Kluczowe cechy

METADATA STRUCTURE

Three-layer tree: metadata files → manifest lists → manifest files (Avro)

★ PARTITION EVOLUTION

Zmień partitioning BEZ przepisywania danych!

MULTI-ENGINE SUPPORT

Spark, Flink, Trino, Snowflake, DuckDB - all first-class citizens

★ HIDDEN PARTITIONING

Użytkownik odpytuje kolumny np. `where date='2026-01-14'`, nie musi znać sposobu partycjonowania

SCHEMA EVOLUTION

Kolumny mają swoje ID i historię w metadanych, można je edytować bez popsucia istniejących zapytań

TIME TRAVEL

Snapshot-based - każdy commit tworzy immutable snapshot

REST CATALOG STANDARD

Vendor-neutral catalog interoperability. Apache Polaris, Nessie, AWS Glue, Hive Metastore.

APACHE ICEBERG

Mocne strony i ograniczenia

✓ MOCNE STRONY

Widest multi-engine support

Partition evolution - unique, bez rewrite

Hidden partitioning - user-friendly

Industry momentum - AWS, Snowflake, DBX

✗ OGRANICZENIA

Metadata overhead - wymaga maintenance

Commit rate limits - ~15 TPS

Catalog required - Hadoop unsafe

Not for high-frequency upserts

BEST FOR

Multi-engine environments • Avoiding vendor lock-in • New lakehouse projects

Startujesz nowy projekt lub chcesz flexibility? → Iceberg

TABLE FORMAT 4 - NEW!

DuckLake

Metadata w SQL database, nie w plikach

KIEDY

May 2025

v0.3 - experimental

KTO

Hannes Mühleisen

DuckDB Labs

STATUS

 **EXPERIMENTAL**

DuckDB native, MotherDuck managed

Spark support coming (funded)

RADYKALNE PYTANIE - DUCKLAKE MANIFESTO

"Dlaczego metadata trzymamy w plikach?"

Iceberg/Delta powstały gdy unikanie baz danych = feature

Ale potem dodaliśmy katalogi.. A katalogi to w sumie bazy danych.

Zatoczyliśmy koło, ale z metadanymi overhead.

→ **Rozwiązanie: Metadata w SQL database od początku**

DUCKLAKE

Kluczowe cechy

★ SQL DATABASE FOR METADATA

DuckDB, PostgreSQL, MySQL, SQLite, not files on storage

★ MULTI-TABLE TRANSACTIONS

BEGIN, update table A, update table B, COMMIT - nie możliwe w Iceberg/Delta

INSTANT QUERY PLANNING

SQL indexes zamiast manifest scanning, claimed 10-100x faster metadata ops

SNAPSHOTS AS ROWS

Millions of snapshots? DB handles trivially. No file cleanup needed.

DATA INLINING

Tiny writes stored inline in catalog, elegantly solves small file problem

ZERO MAINTENANCE

No orphan files, no manifest cleanup, DB referential integrity

PARADIGM SHIFT

Iceberg: call catalog → fetch metadata file → parse → fetch manifests → parse → plan query. DuckLake: one SQL query with indexes.

DUCKLAKE

Mocne strony i ograniczenia

✓ MOCNE STRONY (CLAIMS)

10-100x faster metadata ops (SQL indexes)

Multi-table transactions - finally possible

Nie produkuje wielu małych plików - snapshot = wiersz

Simpler stack – nie wymaga oddzielnego katalogu

✗ OGRANICZENIA

EXPERIMENTAL - not production-ready

Mała adopcja, nawet w benchmarkach vs Iceberg/Delta w dużej skali

DuckDB – wsparcie dla Sparka

Niekompatybilny z innymi formatami – na razie

CO ROBIĆ?

Obserwować • Ocenić pod koniec roku • Nie opierać produkcji

Architektura elegancka, ale potrzebuje dojrzałości

Porównanie formatów: Przegląd funkcji

Funkcja	Apache Iceberg	Delta Lake	Apache Hudi	DuckLake
Aktualna wersja	1.10.1 (Gru 2025)	4.0.1 (Sty 2026)	1.1 (2025)	0.3 (eksperyment.)
Storage metadanych	3-warstwowe drzewo	TX log (JSON+Parquet)	Timeline (akcje)	Baza SQL
Zakres transakcji	Jedna tabela	Jedna tabela	Jedna tabela	Multi-table ACID
Ewolucja schematu	ID-based tracking ✓	Pełne wsparcie ✓	Pełne wsparcie ✓	SQL DDL ✓
Partycjonowanie	Ukryte + ewolucja	Tradycyjne + Liquid	Tradycyjne (rewrite)	Ukryte + ewolucja
Time Travel	Snapshot-based	Wersja/timestamp	Timeline instants	Snapshots jako wiersze
Delete/Update	Positional deletes, DVs	CoW domyślnie, DVs	CoW lub MoR (wybór)	Deletion vectors
Maintenance	Manualny (compaction)	Auto-optimize (DBX)	Auto async compaction	Minimalny (wiersze)
Unikalna cecha	Multi-engine support	Spark + UniForm	Streaming upserts	Multi-table TX

Wszystkie formaty obsługują transakcje ACID, wymuszanie schematu i time travel. Różnice dotyczą podejścia do implementacji i optymalizacji.

Porównanie formatów: Wsparcie silników

Silnik	Iceberg	Delta	Hudi	DuckLake
Apache Spark	✓	✓ Natywny	✓ Natywny	~ W drodze
Trino / Presto	✓ Natywny	✓	✓	-
Apache Flink	✓	✓	✓ Natywny	-
DuckDB	✓	✓	-	✓ Natywny
Snowflake	✓ Natywny	~ UniForm	-	-
StarRocks	✓ Natywny	✓	✓	-
Databricks	✓ (Tabular)	✓ Natywny	✓	-

KLUCZOWY WNIOSEK

Iceberg ma najszersze wsparcie silników

Każdy główny silnik traktuje Iceberg jako first-class. Dlatego vendorzy chmurowi stawiają na ten format.

Mosty konwersji (interoperacyjności)

UniForm - Delta → metadane Iceberg/Hudi

XTable - Dowolny format ↔ Dowolny format

DuckLake – Eksperymentalny eksport metadanych Iceberg (v0.3)

PODSUMOWANIE

Wybór formatu staje się mniej krytyczny. Warstwy interoperacyjności pozwalają czytać dowolny format z dowolnego silnika.

CZĘŚĆ 4

Benchmarki i wydajność

Twarde, wcale nie zmanipulowane dane

DuckLake: Metadane w bazie SQL

TRADYCYJNE PODEJŚCIE (ICEBERG/DELTA/HUDI)

Każde zapytanie wymaga wielu HTTP round-trips do S3 żeby zrozumieć stan tabeli.
Przy dużych tabelach = tysiące plików do skanowania. $O(n)$ complexity.

PODEJŚCIE DUCKLAKE

Metadane w bazie SQL (Postgres, MySQL, DuckDB). Query planning = jedno zapytanie SQL z indeksami. $O(\log n)$ complexity.

10-100× szybsze operacje na metadanych

UNIKALNE MOŻLIWOŚCI

Multi-table transactions - BEGIN, update A, update B, COMMIT

100+ concurrent writers bez konfliktów (vs ~10 w Iceberg)

Brak cleanup - snapshoty to wiersze w bazie

⚠ UWAGI

Wersja 0.3, eksperymentalna

Tylko DuckDB (Spark w drodze)

Nieprzetestowane na petabyte scale

DuckDB: Nowa definicja skali single-node

CLICKBENCH PAŹDZIERNIK 2025

#1 wśród open-source dla hot runs

MotherDuck Mega - najszybszy w chmurze

KOSZT VS SNOWFLAKE

~7× lepsza efektywność kosztowa

MotherDuck \$3.20/h vs Snowflake Small \$4/h

MotherDuck Standard \$1.80/h = Snowflake XL \$32/h

V1.4 LTS (2025)

19% szybsze zapytania • **45% lepsza** tail latency

DLACZEGO DUCKDB JEST INNY?

Columnar + Vectorized - dane w cache CPU

Disk spill na SSD - nie crashuje przy dużych danych

Zero dependencies - jedna paczka (binary), wszędzie działa

Automatic parallel - wykorzystuje wszystkie rdzenie procesora automatycznie

NAJLEPSZY DLA

Development • Embedded analytics • Single-node • CI/CD pipelines

DuckDB: Skala na sprzęcie konsumenckim

27TB TPC-H NA JEDNEJ INSTANCJI AWS

1.19 godziny

Jedna maszyna. Bez klastra. 27 terabajtów.

SPRZĘT KONSUMENCKI

Raspberry Pi 5 (\$300) - TPC-H SF100 w 11.7s

Smartfon Samsung - ten sam benchmark < 4 minuty

SEKRETNA BROŃ: DISK SPILL

Tradycyjne podejście: Dane > RAM → Out of Memory → Crash

DuckDB: Dane > RAM → Spill na SSD → Kontynuuj (wolniej, ale bez crasha)

PRAKTYCZNE ZASTOSOWANIA

Development lokalnie

CI/CD na standardowych GitHub runners

Edge analytics na IoT

Browser analytics via WASM

Benchmarki formatów: Uczciwy obraz

UC BERKELEY LHBENCH (TPC-DS 3TB)

Delta Lake: Baseline (1.0×)

Apache Hudi: 1.4× wolniejszy

Apache Iceberg: 1.7× wolniejszy

⚠ METODOLOGIA MA ZNACZENIE

Onehouse przeanalizował benchmark "Databeans". Po poprawieniu konfiguracji: Delta i Hudi w granicach 5-6% różnicy.

AWS EMR ICEBERG (LISTOPAD 2025)

4.5× szybsze odczyty • **2×** szybsze zapisy • **13.3×** szybsze delete

COW VS MOR - TRADE-OFF

Copy-on-Write: Zapisy wolniejsze (rewrite), odczyty szybkie

Merge-on-Read: Zapisy 1.3-1.4× szybsze, odczyty 3.2× wolniejsze

PRZYKŁAD Z ŻYCIA: SMALL FILES

Tabela z 1,800 małymi plikami: **45 sekund**

Po compaction do 2 plików: **3 sekundy**

Porównanie silników: TPC-DS 1TB na Iceberg

CZAS WYKONANIA (SEKUNDY)



StarRocks vs Trino: 6.93× szybszy | StarRocks vs OSS Spark: 16.6× szybszy

AWS EMR VS OSS SPARK (TPC-DS 3TB)

4.5× szybszy EMR

Koszt EMR: \$4.81

Koszt OSS Spark: \$17.65

WALIDACJA PRODUKCYJNA

Tencent Games: StarRocks + Iceberg, petabytes

Netflix: 15+ klastrów Trino, 10M+ zapytań/mies.

Wybór silnika może zredukować koszty o 60-80%

Benchmarki są syntetyczne

CZEGO TPC-DS I CLICKBENCH NIE TESTUJĄ

Twoja dystrybucja danych - skewed keys, hot partitions

Twoje wzorce zapytań - ad-hoc, dashboardy, UDFs

Concurrent workloads - 100 userów na jednym klastrze

Mixed read/write - streaming + analytics jednocześnie

CO NAPRAWDĘ OSZCZĘDZA PIENIĄDZE

Right-size compute - nie 100 nodów dla zapytania na laptopa

Dopasuj silnik do workloadu - interactive vs batch

Testuj na SWOICH danych - PoC przed commitem

KLUCZOWY WNIOSEK

Największe oszczędności daje **wybór silnika**, nie formatu.
Różnice 6-18× między silnikami na tych samych danych.

Przewodnik wyboru silnika

DuckDB

Najlepszy dla: Development, embedded, single-node < 100GB

Disk spill, zero dependencies, WASM

StarRocks

Najlepszy dla: Dashboardy, ad-hoc SQL, sub-second latency

Vectorized MPP, petabyte scale

Trino

Najlepszy dla: Cross-source queries, federacja, SQL na wszystkim

100+ connectorów, data mesh

Spark

Najlepszy dla: ETL, transformacje, ML pipelines, batch

MLlib, GraphX, Structured Streaming

Flink

Najlepszy dla: Real-time, event-driven, CDC pipelines

True streaming, exactly-once

Większość organizacji potrzebuje **więcej niż 1 silnika**. Otwarte formaty pozwalają używać właściwego narzędzia do każdego zadania bez duplikacji danych.

CZĘŚĆ 5

Trendy rynkowe 2026

Dokąd zmierza ekosystem?

Wojna formatów skończona: Iceberg wygrał

AWS (Grudzień 2024): S3 Tables z natywnym wsparciem Iceberg, REST Catalog APIs

Snowflake (Lipiec 2024): Open-source Polaris Catalog dla Iceberg, donacja do Apache

Databricks (Czerwiec 2024): Akwizycja Tabular (twórcy Iceberg) za >\$1B

Microsoft: Fabric z XTable dla Delta/Iceberg |

Google: BigLake wspiera Iceberg

DELTA LAKE NIE UMARŁ

10,000+ firm w produkcji

60%+ Fortune 500 używa Databricks

Głęboko zakorzeniony w ekosystemie Spark

PRAWDZIWA ZMIANA

Dyskusja przeszła z **"Który format?"** na **"Który katalog?"**

Interoperacyjność formatów rozwiązana. Katalog = nowe pole bitwy.

Zaczyna się wojna o katalogi

REST Catalog

Otwarty standard (Netflix). Vendor-neutral, przenośny między chmurami.

Implementacje: Polaris, Nessie, S3 Tables

+ Maksymalna elastyczność i przenośność

Unity Catalog

Databricks. Unified governance, row-level security, lineage, AI/BI integracja.

OSS wersja istnieje, ale z mniejszą liczbą features

+ Głęboka integracja jeśli jesteś na Databricks

AWS Glue

Integracja z ekosystemem AWS. Athena, EMR, Redshift Spectrum, Lake Formation.

AWS-native = ból przy migracji do innej chmury

+ Jeśli jesteś all-in na AWS, po prostu działa

Dlaczego to ważne: Katalog determinuje które silniki mają dostęp, governance capabilities, i trudność migracji. Konwersja formatów jest łatwa - migracja katalogów trudna.

Konwergencja: Wybór formatu mniej istotny

Delta Lake UniForm

Delta 3.0 (Październik 2023). Zapisujesz Delta, automatycznie generowane metadane Iceberg/Hudi.

Ograniczenie: Read-only dla nie-Delta klientów

Apache XTable (Incubating)

Omni-directional konwersja między DOWOLNYMI formatami. Iceberg ↔ Delta ↔ Hudi.

v0.3.0 (Czerwiec 2025). Pełny read/write.

Microsoft Fabric

"Table format virtualization" przez XTable. Tabele Delta widoczne jako Iceberg i odwrotnie. Bez duplikacji danych.

MATRYCA INTEROPERACYJNOŚCI

UniForm: Delta → inne (read-only)

XTable: Dowolny ↔ dowolny (full read/write)

DuckLake v0.3: DuckLake → Iceberg (eksport metadanych)

GDY KONKURENCI WSPÓŁPRACUJĄ...

Kontrybutorzy XTable: Microsoft, Google, Databricks, Snowflake, Walmart, Adobe, Cloudera, Dremio.

Kiedy rywale zgadzają się na standard - to sygnał.

Predykcje 2026: Co nas czeka?

1. **Konsolidacja katalogów:** REST Catalog jako standard. Apache Polaris zyska więcej adopcji.

2. **DuckDB wchodzi do enterprise.** DuckLake dostanie wsparcie Spark. Produkcyjne wdrożenia.

3. **AI/ML natywne:** Vector embeddings domyślnie obsługiwane. AI governance w katalogach.

4. **Wzrost Streamingu:** Duża świeżość danych jako oczekiwanie, nie wyjątek.

5. **Formaty Tabel łatwo wymienne:** UniForm i XTable dojrzewają. "Który format?" przestaje być pytaniem.

WIELKOŚĆ RYNKU

\$66-112 mld

do 2030-2035 (szacunki)

OCENA FORRESTER

"Widzimy przyszłość warehouses i lakes schodzących się w lakehouse, gdzie jeden system jest wystarczający."

Architektura lakehouse wygrywa.

Pytanie: którą implementację i ekosystem wybierzesz?

CZĘŚĆ 6

Pytania?



CZĘŚĆ 6

Które podejście do katalogu? Liniowe, Drzewowe czy Bazowe?

CZĘŚĆ 6

Czym się różni lakehouse od modern data warehouse?

CZĘŚĆ 6

Do czego silnik Spark, a do czego DuckDB?